

Basic Analysis in Lean4: A Walk Through

James Squires

December 2025

This paper is meant to provide a basic understanding and intuition of what Lean encodings of mathematics, particularly in real analysis, look like. We will explain the basic theory behind propositions in Lean. Then we will apply our understanding to produce the Completeness Axiom and some basic lemmas from the Completeness Axiom.

1 Lean Structure

Lean4 operates on a formal logic called Martin-Löf type theory. This logic uses terms and types as mathematical objects, and manipulates them via inference rules to produce what are called judgments. Fortunately, we don't need to understand all of this logic to do mathematics in Lean, just like how mathematicians don't need to fully understand formal logic like syntax and semantics of first order logic to do mathematics (although they still need a strong understanding of how to use first order logic). Instead we can focus on how to naturally code (prove) propositions in Lean.

1.1 Propositions

Proofs of propositions (`lemmas`, `theorems`, `examples`) in Lean require three things: a context, the statement to be proved (the proposition itself), and the proof. Here's how a lemma in Lean looks:

Listing 1: Propositions in Lean

```
lemma Name_of_Lemma (Context) : Propositional_Statement :=  
  Proof
```

This proposition starts with `'lemma'`. This is just us telling Lean to expect a proposition. Then we name our proposition `'Name_Of_Lemma'` so that we can use it in future code. Then we write our context. Think of our context like our assumptions or hypothesis (what we already know). A context can be empty. Afterward we write `':'` and follow this by our propositional statement. The `':'` simply separates our context from our proposition (it tells Lean to now expect the proposition). We write `':='` after our proposition to mark the beginning of our proof. Then we do the proof, which should ultimately demonstrate that

we can produce the proposition from our context.¹ Here's a more concrete example²:

Listing 2: Lemma: AddOneBothSides

```
lemma Add_One_Both_Sides (a b : Int) (h : a = b) : a + 1 = b + 1 := by
  rewrite [h]
  rfl
```

Let's break down what the Lean code does here.

1. First we state our lemma named 'Add_One_Both_Sides'
2. Then we state our context: (a and b are both integers) AND (we have a hypothesis called 'h' that says a = b)
3. After the colon we wrote our proposition/goal: $a + 1 = b + 1$
4. Once we want to begin the proof, we write ':='.³
5. We perform our proof using two separate tactics: `rewrite` and `rfl`

The most difficult part about Lean is understanding the *proof* language. There's an underlying type-theoretic based logic that the computer is using to *type check* or confirm our propositions as true. However, we can avoid much of this formal logic (which often gets unwieldy rather quickly) by using what are called tactics. A tactic is just a more natural/human way of proving things. A tactic implicitly uses these logic manipulations to transform our goal state (similar to how we rewrite what we intend to prove in classical math).

In our lemma above, we use the word 'by' to tell Lean that we're now going to make our proof using tactics. Once we've written our proof with tactics, Lean will process the proof into its underlying logic and computationally check our code. After we write 'by' in Listing 2, we then use our first tactic:

`rewrite [h]`

`rewrite` uses our hypothesis called h, which states $a = b$, to substitute a for b in all occurrences of a in our previously stated proposition/goal. So using hypothesis $h : a = b$:

$$a + 1 = b + 1 \implies (b) + 1 = b + 1.$$

Then we use our second tactic called '`rfl`', which stands for reflexivity. This seals the proof; once we've applied reflexivity, because $b + 1 = b + 1$, we've arrived at a statement we know to be true. Think of this step as similar to when we want to prove 2 is a natural number using Peano's axioms. We

¹In Martin-Lof Type Theory, our proposition is a type. When we give a proof, we're giving a term that inhabits the type. If we can prove this term inhabits the type by constructing the term, then we've derived our proposition.

²Lean tells the user if their proposition is correct; all of the propositions I'm showing have already been type checked for correctness.

rewrite our statement to 0 is a natural number, and we already know this to be true, therefore our proof holds. Likewise we reduce our statement in Lean to something that is trivially true using our tactic for reflexivity. Once we've done this, Lean checks the proof and tells us the proof is correct.

2 Definitions: Completeness Axiom and Open Intervals

We will demonstrate more aspects of Lean by trying to prove $(-\pi, \pi)$ has a least upper bound.

2.1 Axiom of Completeness

We want to define the following axiom in Lean4:

$$\forall s \subset \mathbb{R}, s \neq \emptyset, s \text{ bounded above} \implies (\exists x \in \mathbb{R}, x = \text{lub}(s)) \quad (1)$$

We don't need to provide a proof, rather we're just going to provide an axiom, a proposition which we treat as true. The structure in Lean is the following:

Listing 3: Axioms in Lean

```
axiom Name_Of_Axiom :  
  Proposition
```

A concrete example using the Completeness Axiom looks like the following:

Listing 4: Stating the Axiom of Completeness

```
axiom completeness :  
   $\forall (s : \text{Set } \mathbb{R}), s.\text{Nonempty} \rightarrow \text{BddAbove } s \rightarrow \exists x : \mathbb{R}, \text{IsLUB } s \ x$ 
```

We'll read this in steps.

1. $[\forall (s : \text{Set } \mathbb{R})]$ says for all subsets s in the real numbers
2. $[s.\text{Nonempty} \rightarrow \text{BddAbove } s \rightarrow \exists x : \mathbb{R}, \text{IsLUB } s \ x]$ says both s is nonempty AND s is bounded above implies there is an x in the real numbers such that x is the least upper bound of s .³

Here 'IsLUB' is a function that takes a set 's' and the least upper bound of s, which we called x, and returns the proposition that x is the least upper-bound of s.

³In classical logic, $P \implies (Q \implies R)$ is equivalent to $(P \wedge Q) \implies R$. Lean associates arrows to the right. We can think of these arrows as implications here.

2.2 Definition of $(-\pi, \pi)$

Now we're going to build our definition of $(-\pi, \pi)$ in Lean. To do this we will use a Lean function 'def' to define our term (our mathematical object). When we define our term in Lean, we also give it a type. We've already implicitly done this above. For example, when we wrote $(s : \text{Set } \mathbb{R})$, we gave the term 's' a type 'Set $\mathbb{R}$ '. There aren't really many classical examples of types and terms, but think of these as closely analogous to elements and sets. Just like how a term inhabits a type, an element is inside a set. The general structure of a Lean definition goes as following:

Listing 5: Definitions in Lean

```
def Name_Of_Def : Type_Of_The_Term :=  
  Term
```

We tell Lean that our name ends and our type begins by typing a colon. And after the type, we tell Lean to expect our term by typing ':='. We will now produce a concrete example of a Lean definition below:

Listing 6: Defining the open interval $(-\pi, \pi)$

```
def openPi : Set  $\mathbb{R}$  :=  
  Set.Ioo (-Real.pi) Real.pi  
-- (This is a comment in Lean) Real.pi is how we write out pi from the  
   real numbers
```

1. `openPi` gives the name of what we're defining
2. `Set  $\mathbb{R}$` tells Lean to expect the mathematical object (type) we're defining to be a subset of the reals.
3. `Set.Ioo` is a function that takes in two numbers `m` and `M` and returns the open subset (m, M)

The open subset we gave is a term of the type that we told Lean to expect, so our definition `openPi` is well defined (we've successfully defined $(-\pi, \pi)$).

3 Theorems in Analysis: Proving $(-\pi, \pi)$ Has a Least Upper Bound

3.1 Nonemptiness of $(-\pi, \pi)$

If we want to prove that `openPi` has a least upper bound, we first have to prove each part of the left side of our Completeness Axiom implication. First we will prove `openPi` is nonempty. We will do this by showing $0 < \pi$ and $-\pi < 0$. The code is as follows:

Listing 7: Proving $(-\pi, \pi)$ is nonempty

```
lemma openPi_nonempty : openPi.Nonempty := by
  use 0
  constructor
  . exact neg_lt_zero.mpr Real.pi_pos
  . exact Real.pi_pos
```

1. We name our lemma `openPi_nonempty`.
2. Our lemma is the proposition: `[openPi.Nonempty]`
3. `[openPi.Nonempty]` is a function that takes our subset `openPi` and gives the proposition:

$$\exists x, x \in \mathbb{R}, (-\pi < x) \wedge (x < \pi)$$

4. Our first tactic, `[use]`, tells Lean to substitute 0 for x in our proposition and try to prove:

$$\exists 0, (-\pi < 0) \wedge (0 < \pi)$$

5. `[constructor]` tells Lean to break our conjunctive (and) statement into cases, so instead we now need to prove two subgoals:

- (a) $(-\pi < x)$
- (b) $(x < \pi)$

6. `exact A` tells Lean to check whether A satisfies the given goal. In this case we're using `exact` to check two lemmas against our goals 5.(a) and 5.(b).

7. We use two already proven lemmas (we can import these from online code):

- (a) `[neg_lt_zero.mpr 0 < a]`: this lemma says $0 < a \implies -a < 0$
- (b) `[Real.pi_pos]`: this lemma says $0 < \pi$

8. So `[neg_lt_zero.mpr Real.pi_pos]` takes `Real.pi_pos`, which says $0 < \pi$, and gives $-\pi < 0$ to be true

9. Putting it all together, `[exact neg_lt_zero.mpr Real.pi_pos]` says our subgoal is satisfied (the lemma we supplied matches the goal)

10. Likewise, `[exact Real.pi_pos]` says our second subgoal is satisfied

3.2 Boundedness Above

Listing 8: Proving that $(-\pi, \pi)$ is bounded above

```
lemma openPi_bddAbove : BddAbove openPi :=
  < Real.pi, by
  intro x hx
  exact hx.2.le >
```

1. BddAbove S is a function that says a set S is bounded above. We write functions like f(x) as f x in Lean. `BddAbove openPi` states

$$\exists M, \forall x \in \text{openPi}, x \leq M$$

Or equivalently

$$\exists M, x \in \text{openPi} \rightarrow x \leq M$$

2. We supply π and a proof of π as an upper bound: $\langle \pi, \text{Proof of BddAbove openPi} \rangle$. The brackets simply state that π (the left side) is being used for the proof (the right side)
3. Now that we supplied π , we have a new goal inside the brackets. We have to prove $x \in \text{openPi} \rightarrow x \leq \pi$
4. We prove BddAbove openPi using the tactics `intro` and `exact`
 - (a) `intro x hx` tells Lean to assume the hypothesis hx, which says $x \in \text{openPi}$.⁴
 - (b) `[hx.2.le]` is simply the lemma $x \leq \pi$ (this follows from: if x is in the set, then $x < \pi$. If $x < \pi$, then $x \leq \pi$).^{5 6}
 - (c) `[exact hx.2.le]` tells Lean to check whether $x \leq \pi$ matches our stated goal, which it does.

It's easy to get lost in the Lean logic here, but what we've done is said: take π as our upper bound. Assume x is in `openPi`. If it is, then $x \leq \pi$. But the definition of being BddAbove by π is $x \leq \pi$. Thus if we assume x is in `openPi`, then $x \leq \pi$.

⁴If we need to prove $P \implies Q$, and we assume P, then Q proves $P \implies Q$ (*Modus Ponens*)

⁵hx is equivalently $-\pi < x \wedge x < \pi$. hx.1 is the left side of our and statement while hx.2 is the right hand side.

⁶ $(a < b).le$ is the lemma $(a < b) \implies (a \leq b)$

3.3 Existence of a LUB via Completeness

We have finally proven all our supporting lemmas. To prove we have a least upper bound on $(-\pi, \pi)$, all we have to do is sequentially apply our lemmas to our axiom:

Listing 9: Applying the completeness axiom to obtain the LUB

```
theorem openPi_has_lub :  
   $\exists x : \mathbb{R}, \text{IsLUB openPi } x :=$   
  completeness openPi openPi_nonempty openPi_bddAbove
```

`theorem` works exactly the same as `lemma`. The proof might look a bit strange, but it's actually quite simple. Our completeness axiom takes three arguments: a set `s`, the lemma that `s` is non-empty, and a lemma that `s` is bounded above. Think of this as a function $f(A, B, C)$, but instead it's written as `f A B C`⁷. Thus we've proven $(-\pi, \pi)$ has a least upper bound and we're done!

4 Conclusion

It might seem like we've done a lot of work to prove something quite small. This is because learning Lean is learning a new language. The reward is immense though. Theorems proven in Lean cannot be contested for their validity. For long and involved proofs, especially with many cases, Lean catches all logic/computational mistakes and streamlines the review process. It also revolutionizes collaboration in mathematics. A problem can be broken up into many parts and delegated across the world to various mathematicians of different skill sets. These mathematicians can then combine their code and together produce incredibly challenging proofs in record time. A famous example of this is the Liquid Tensor Experiment which required hundreds of thousands of lines of code and many collaborators. In between Lean's facilitation of collaboration and workflow, and its ability to verify AI generated math proofs for correctness, this has solidified Lean's place in the future of mathematics. I encourage further exploration by exploring both type theory and Lean4 coding guides. Some good resources are *Type Theory and Formal Proof: An Introduction* by Herman Geuvers and Rob Nederpelt and *Theorem Proving in Lean4* (listed on github).⁸ Additionally, a sizable amount of modern mathematics proved via Lean4 can be found on <https://github.com/leanprover-community/mathlib4>.

⁷This is called Currying

⁸No citations needed.